

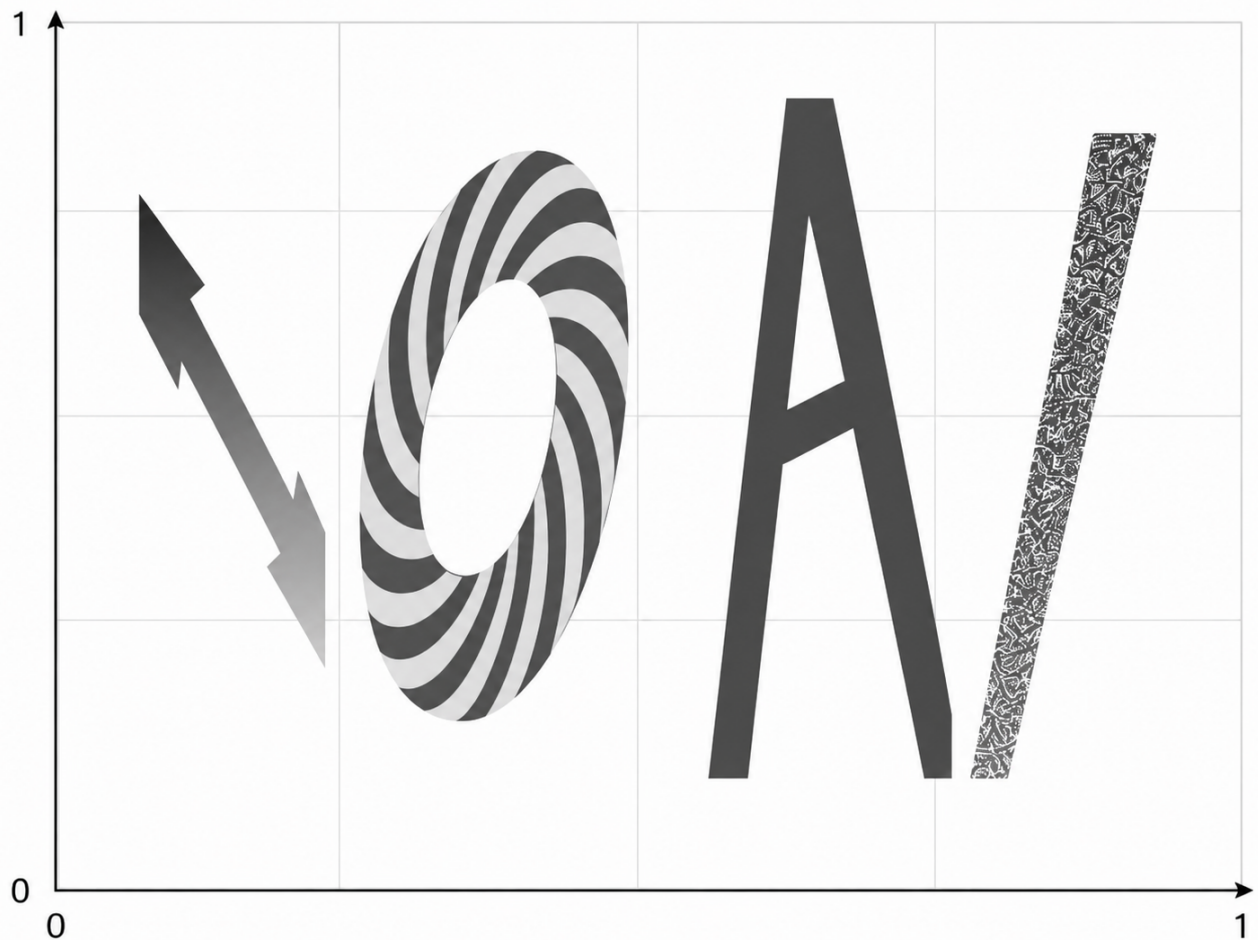
Champ IOAI

- **Limite de temps** : 5 minutes
- **Stockage** : 5 GB
- **Taille de la solution** : `solution.ipynb`, `custom_model.py` ≤ 1 MB au total
- **Modèles préentraînés** : aucun — entraînement à partir de zéro, sans internet lors de l'évaluation
- **Score baseline** : 31.2187

Tâche

Le maire d'Astana souhaite décorer la ville avec des logos IOAI stylisés. En tant que statisticien, il considère tout — y compris le logo — comme une fonction spatiale $F(x, y, \overline{W})$, où $x, y \in [0, 1]$ représentent les coordonnées dans un plan 2D et $\overline{W}$ est un ensemble de paramètres cachés définissant des attributs stylistiques tels que les couleurs et les angles des lettres.

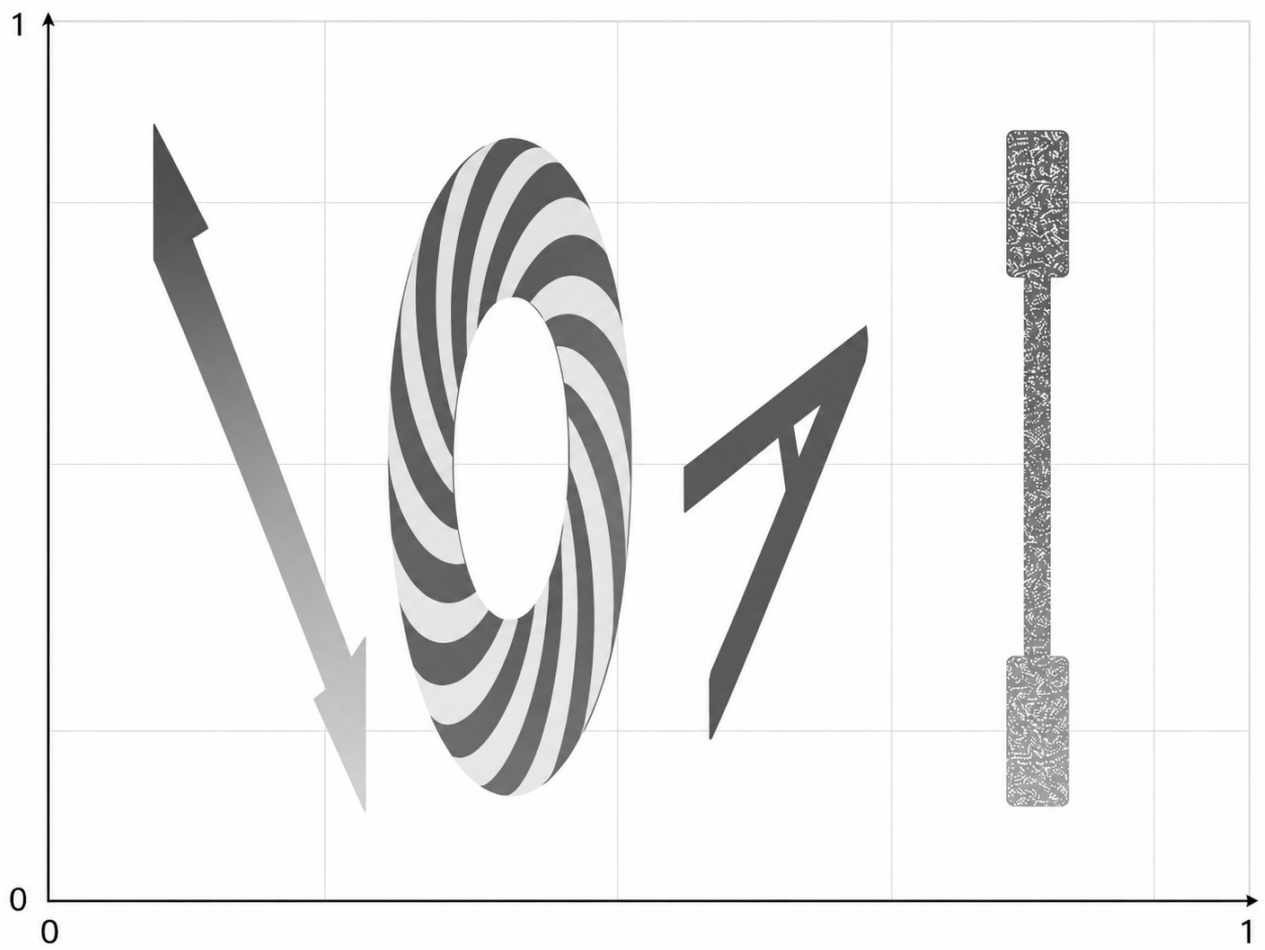
Comme F est trop complexe pour être exprimée par une équation mathématique explicite, votre tâche consiste à entraîner un réseau de neurones pour l'approximer. Le réseau produira une valeur de **champ IOAI (IOAI Field)** pour toute paire de coordonnées (x, y) , générant une visualisation complète du logo sous forme de carte thermique (heatmap) dans tout le plan. Voici un exemple de visualisation sous forme de carte thermique de F avec certains paramètres cachés spécifiques $\overline{W}$.

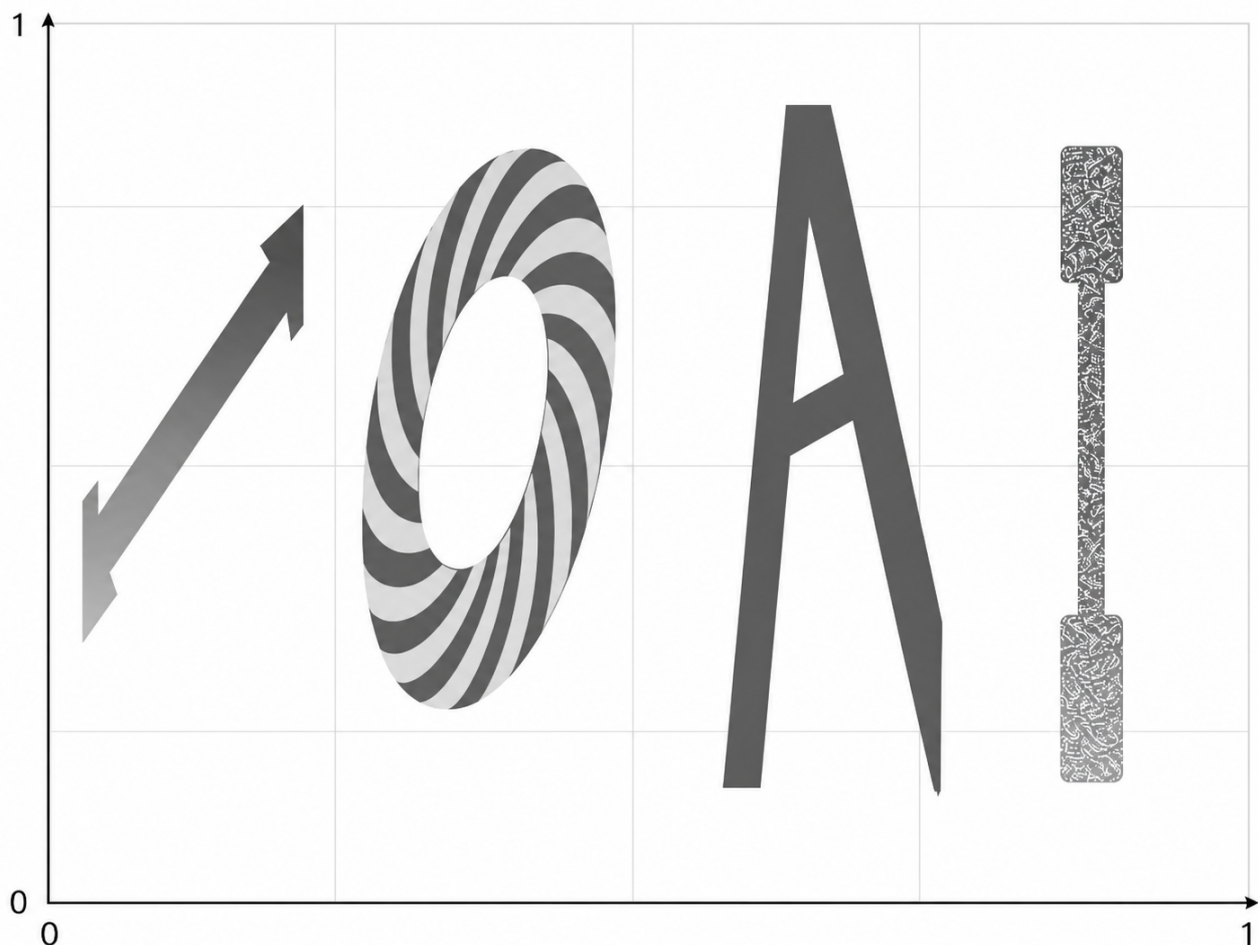


De quoi le champ IOAI est-il constitué ? De quatre lettres et de l'arrière-plan.

- Les valeurs à l'intérieur de la première lettre $\mathbb{I}$ sont très grandes ($1e+10$ et plus), avec un gradient linéaire
- Les valeurs dans la lettre $\mathbb{O}$ présentent un motif en spirale
- La valeur à l'intérieur de la lettre $\mathbb{A}$ est toujours -1
- Les valeurs à l'intérieur de la dernière lettre $\mathbb{I}$ doivent être des valeurs aléatoires appartenant à l'intervalle $[-2026, 2026]$, même si elles sont évaluées deux fois au même point
- En dehors des lettres, la valeur est toujours nulle

La fonction possède des paramètres cachés $\overline{W}$, qui influencent l'échelle et l'inclinaison des lettres, ainsi que l'intervalle des valeurs à l'intérieur de la première lettre $\mathbb{I}$. Cependant, les lettres ne se chevaucheront pas. Voici quelques exemples illustrant l'apparence du champ IOAI avec différentes valeurs de $\overline{W}$:





Ce qui vous est fourni :

Ce problème ne contient AUCUN dataset. À la place, la fonction génératrice vous est fournie et est configurée par le fichier de configuration JSON situé à `data/train_config/field_config.json`.

La configuration de test est cachée, mais elle est de nature similaire. Votre tâche consiste à ajuster (fit) votre modèle sur le générateur fourni en utilisant autant de données que vous le souhaitez. Vos distributions « train » et « test » sont générées par le même générateur — vous ne savez simplement pas sur quels points (x_i, y_i) vous serez évalué.

Votre soumission doit comprendre :

- la classe du modèle entraîné, enregistrée sous `custom_model.py`. Ce modèle doit hériter de la classe `torch.nn.Module` et utiliser uniquement les imports `torch`. Il doit contenir la classe `CustomModel` utilisée dans le notebook `solution.ipynb`.
- le notebook `solution.ipynb`, qui produira les poids `model.pt`

Évaluation

Pour chaque région, le score minimal est 0 et le score maximal est 1. Le score final est la moyenne des scores des cinq régions (une pour chacune des quatre lettres et une pour l'arrière-plan), multipliée par 100. Il existe une **pénalité liée au nombre de paramètres** :

Si votre modèle possède plus de 20260 paramètres, le score est divisé par deux.

Le nombre de paramètres est mesuré par `sum(p.numel() for p in model.parameters())`. Nous attendons également de votre modèle qu'il fonctionne en mode stochastique, avec `nn.Dropout` de PyTorch intégré au modèle.

Pour les régions standard

Pour chaque région R (première lettre `I`, `O`, `A`, `Background`), nous évaluons le modèle sur $N_R = 512$ points de test (x_i, y_i) , avec les valeurs réelles v_i et les prédictions $\hat{v}_i$. Nous utilisons l'erreur absolue moyenne normalisée (MAE - Mean Absolute Error) comme métrique principale. La MAE est définie comme suit :

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Et la normalisation est effectuée comme suit :

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

où $s_R > 0$ est une constante d'échelle.

Pour la région de la dernière lettre `I`

Dans cette région, **le dropout est activé pendant l'évaluation**. Pour chaque point de test j :

1. Nous exécutons le modèle $K = 10$ fois afin d'obtenir $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Si une sortie quelconque se trouve en dehors de l'intervalle $[-2026, 2026]$, alors $\text{pointScore}(j) = 0$.
3. Sinon, nous calculons l'écart-type (standard-deviation) σ_j des K sorties (outputs) et le convertissons en score :

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

où $s_E > 0$ est une constante d'échelle fixe.

Le score de la région est la moyenne sur tous les points de la région :

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

où $N_E = K * N_R$.

En termes simples, plus la diversité est grande, plus votre score pour cette région sera élevé. **Vous ne pouvez pas utiliser directement de valeurs aléatoires, notamment les fonctions `PyTorch rand*` et `_uniform`; le caractère aléatoire doit provenir de l'inférence avec le dropout activé.**

Procédure de soumission

1. Ouvrez `solution.ipynb` et exécutez toutes les cellules.
2. Améliorez le modèle `CustomModel` dans `custom_model.py`
3. Assurez-vous que votre dernière cellule enregistre votre modèle dans le fichier `model.pt`.
4. Dans l'onglet Git de JupyterLab, ajoutez à l'index, commentez et validez `solution.ipynb` et `custom_model.py`, puis poussez-les.
5. Revenez à la page du concours et cliquez sur **Soumettre**. Le commentaire de soumission doit être identique au commentaire de l'étape précédente.